

Zod

TypeScript-first schema validation
with static type inference

[colinhacks/zod](#) · v4.4.3 · MIT

TypeScript types vanish at runtime

Any data crossing a boundary — API response, env var, form input —
arrives as `unknown` at runtime. Your types aren't there to help.

Without Zod

```
// Types declared separately from runtime checks
type User = {
  id: number;
  email: string;
};

// Guards written by hand — and they drift
function isUser(data: unknown): data is User {
  return (
    typeof data === 'object' && data !== null &&
    typeof (data as any).id === 'number' &&
    typeof (data as any).email === 'string'
  );
}

const raw = await fetch('/api/user').then(r => r.json());
if (!isUser(raw)) throw new Error('unexpected shape');
const user: User = raw; // still "trust me" territory
```

With Zod

```
import * as z from "zod";

// One schema. Types AND validation together.
const User = z.object({
  id: z.number(),
  email: z.string().email(),
});

type User = z.infer<typeof User>;

const raw = await fetch('/api/user').then(r => r.json());

// Validated. Typed. No manual guard.
const user = User.parse(raw);
console.log(user.email); // string — guaranteed
```

Parse untrusted data — get a typed result

```
import * as z from "zod";

// Define a schema
const Player = z.object({
  username: z.string(),
  xp: z.number().nonnegative(),
  role: z.enum(["admin", "player", "spectator"]),
});

type Player = z.infer<typeof Player>;
// { username: string; xp: number; role: "admin"|"player"|"spectator" }

// Valid input → typed deep clone
const player = Player.parse({
  username: "billie", xp: 100, role: "player"
});
// ⇒ { username: "billie", xp: 100, role: "player" }

// Invalid input → structured ZodError
try {
  Player.parse({ username: 42, xp: -5 });
} catch (err) {
  if (err instanceof z.ZodError) {
    err.issues;
    // [{ code: "invalid_type", path: ["username"], ... },
```

Infer static types — no duplication

```
import * as z from "zod";

const Player = z.object({
  username: z.string(),
  xp: z.number(),
});

// Extract the TypeScript type — schema IS the source of truth
type Player = z.infer<typeof Player>;
// ⇒ { username: string; xp: number }

// Transforms give you distinct input and output types
const WithTimestamp = z.string().transform(s ⇒ new Date(s));
type In  = z.input<typeof WithTimestamp>;  // string
type Out = z.output<typeof WithTimestamp>; // Date
```

Change the schema once. The TypeScript type updates everywhere automatically.

safeParse — no try/catch required

```
import * as z from "zod";

const Config = z.object({
  port: z.number().int().min(1).max(65535),
  host: z.string(),
  debug: z.boolean().default(false),
});

// Result is a discriminated union — narrowing is native
const result = Config.safeParse(process.env);

if (!result.success) {
  // result.error is a ZodError
  console.error(result.error.flatten());
  process.exit(1);
}

// result.data is fully typed: { port: number; host: string; debug: boolean }
startServer(result.data);
```

`safeParseAsync` handles async refinements and transforms.

14x

faster string parsing in Zod 4

7x faster arrays · 6.5x faster objects · 100x fewer `tsc` instantiations

Benchmarked against Zod 3 · node v22.13.0 · arm64-darwin

Zod 3 → Zod 4: smaller and faster to compile

Bundle size (gzipped)

TypeScript compilation

Package	Size	Instantiations	
Zod 3	12.47 kb	Zod 3	> 25,000
Zod 4	5.36 kb	Zod 4	~ 175
Zod 4 / mini	1.88 kb	Simple <code>z.object().extend()</code> no longer blows up <code>tsc</code> on large codebases.	

`zod/mini` uses a functional API that tree-shakes to only what you import.

185M

weekly npm downloads

42,878 GitHub stars · 734M monthly downloads

Started at 600k/week when v3 launched in May 2021.
Now in the same download tier as foundational infrastructure libraries.

An ecosystem built on Zod

API & RPC

- tRPC — end-to-end type-safe APIs
- Hono `@hono/zod-validator` middleware
- OpenAPI / JSON Schema generation

Forms

- react-hook-form `zodResolver`
- Valibot (Zod-compatible API surface)

Data & ORM

- Drizzle ORM `drizzle-zod`
- Prisma schema generation

AI / LLM tooling

- Vercel AI SDK structured outputs
- LangChain tool schemas

Libraries you already use speak Zod natively — schema definitions travel across your stack.

zod/mini — 1.88 kb when every byte counts

```
import * as z from "zod/mini";

// Functional API — same parse/safeParse/parseAsync behavior
const User = z.object({
  name: z.string(),
  age: z.optional(z.number()),
  roles: z.array(z.enum(["admin", "viewer"])),
});

type User = z.infer<typeof User>;

// Checks are composable — bundlers tree-shake what you don't import
const PositiveArray = z.array(z.number()).check(
  z.minLength(1),
  z.refine(arr => arr.every(n => n > 0), "all values must be positive"),
);
```

6.6× smaller than Zod 3 · ideal for edge functions, service workers, and tight Lambda budgets

Get started in 30 seconds

```
npm install zod
```

```
import * as z from "zod";

// Validate an API response
const Repo = z.object({
  id: z.number(),
  name: z.string(),
  stargazers: z.number(),
  archived: z.boolean().default(false),
});

type Repo = z.infer<typeof Repo>;

const raw = await fetch("https://api.github.com/repos/colinhacks/zod")
  .then(r => r.json());

const repo = Repo.parse(raw);
console.log(`${repo.name}: ${repo.stargazers} stars`);
// => "zod: 42878 stars"
```

Zero dependencies. Works in Node.js, Deno, Bun, and every modern browser.

One schema. Types that hold
in development and production.

```
npm install zod
```



zod.dev



[colinhacks/zod](https://github.com/colinhacks/zod)



[Discord](#)

Try it on one API boundary this week.